Practical No. 1 : Implementation of A Search Algorithm for the 8 -puzzle problem

```
import heapq

GOAL = (1,2,3,4,5,6,7,8,0)

def h(s):
    return sum(abs(i//3 - GOAL.index(x)//3) +
               abs(i%3 - GOAL.index(x)%3)
               for i,x in enumerate(s) if x != 0)

def astar(start):
    pq = [(h(start), 0, start, [])]
    visited = set()

    while pq:
        f, g, s, path = heapq.heappop(pq)

        if s in visited:
            continue
        visited.add(s)
        path = path + [s]

        if s == GOAL:
            print("Solution Path\n")
            for step, state in enumerate(path):
                print("Step", step)
                for i in range(0, 9, 3):
                    print(state[i:i+3])
```

```
            print("Total Moves =", len(path)-1)
            return

        z = s.index(0)

        for d in [-3, 3, -1, 1]:
            n = z + d

            if 0 <= n < 9 and not (d == -1 and z%3 == 0) and not (d == 1 and z%3 == 2):
                new = list(s)
                new[z], new[n] = new[n], new[z]
                new = tuple(new)

                if new not in visited:
                    heapq.heappush(pq, (g+1+h(new), g+1, new, path))

start = (1,2,3,4,0,6,7,5,8)
astar(start)
```